Caminhos mínimos em grafos dirigidos

Algoritmos de Dijkstra e de Bellman–Ford

J.D.

Universidade Federal do ABC

12 de novembro de 2026

Roteiro

O problema

Algoritmo de Dijkstra

Algoritmo de Bellman–Ford

Comparação

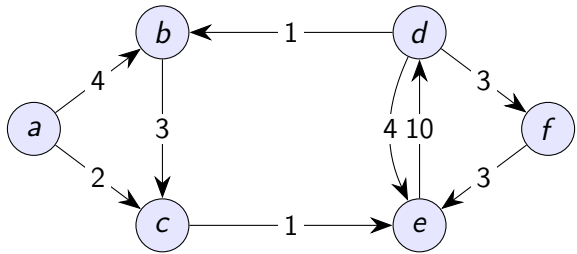
Grafos dirigidos com pesos nos arcos

Um grafo $G = (V, E, \rho)$ com $\rho\colon E \to \mathbb{R}$.

- **Comprimento** de um caminho $P = x_1, \ldots, x_k$: $c(P) = \sum_{i=1}^{k-1} \rho(x_i, x_{i+1})$.
- **Distância**: $\text{dist}_G(u, v) = \min\{c(P) : P \text{ é um } (u \to v)\text{-caminho}\}$, e ∞ se não há caminho.
- Em grafos dirigidos, em geral $\text{dist}(u, v) \neq \text{dist}(v, u)$.

Problema (origem única): dados G e $s \in V$, calcular $\text{dist}_G(s, v)$ para todo $v \in V$.

Um exemplo



Por exemplo, $\text{dist}(c, e) = 1$, mas $\text{dist}(e, c) = 10 + 1 + 3 = 14$.

Duas propriedades que sustentam tudo

Desigualdade triangular

Para todo arco $(v, u) \in E$: $\text{dist}(s, u) \leq \text{dist}(s, v) + \rho(v, u)$.

- Ir até v pelo melhor caminho e depois usar o arco (v, u) é *um* caminho até u; o melhor não pode ser pior.

Subestrutura ótima

Todo trecho de um caminho mínimo é, ele mesmo, um caminho mínimo.

- Se houvesse um atalho dentro do trecho, poderíamos trocá-lo e melhorar o caminho inteiro, o que é absurdo.

Relaxação

Mantemos, para cada vértice v, uma estimativa $\mathrm{d}[v]$ da distância e um predecessor $\mathrm{p}[v]$. A estimativa sempre é o custo de algum caminho real, portanto $\mathrm{d}[v] \geq \mathsf{dist}(s, v)$.

Relaxação(u, t)

```
1 se d[t] > d[u] + ρ(u, t) então
2 |   d[t] ← d[u] + ρ(u, t)
3 |   p[t] ← u
```

Pergunta central: em que ordem relaxar os arcos para que as estimativas se tornem exatas?

Relaxação

Mantemos, para cada vértice v, uma estimativa $\mathrm{d}[v]$ da distância e um predecessor $\mathrm{p}[v]$. A estimativa sempre é o custo de algum caminho real, portanto $\mathrm{d}[v] \geq \mathsf{dist}(s, v)$.

Relaxação(u, t)

```
1 se d[t] > d[u] + ρ(u, t) então
2 |   d[t] ← d[u] + ρ(u, t)
3 |   p[t] ← u
```

Pergunta central: em que ordem relaxar os arcos para que as estimativas se tornem exatas?

- **Dijkstra:** uma ordem esperta, cada arco uma única vez (pesos ≥ 0).
- **Bellman–Ford:** sem ordem especial, repetindo tudo várias vezes (pesos quaisquer).

Dijkstra: a ideia

- Mantemos um conjunto S de vértices já resolvidos, isto é, com $\mathrm{d}[v] = \mathrm{dist}(s, v)$ garantidamente.
- A cada passo, escolhemos em $\overline{S} = V \setminus S$ o vértice u de menor $\mathrm{d}[u]$ e o movemos para S.
- Em seguida, relaxamos os arcos que saem de u.

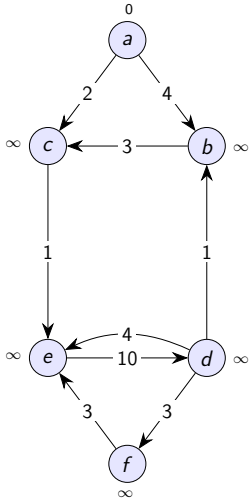
Intuição

Como os pesos são ≥ 0, nenhum desvio por vértices ainda não resolvidos pode ser mais barato do que o vizinho mais próximo da fronteira. Assim, o vértice de menor d já está *definitivamente* correto.

Dijkstra no exemplo (origem $s = a$)

iteração	escolhe u	$\mathrm{d}[a]$	$\mathrm{d}[b]$	$\mathrm{d}[c]$	$\mathrm{d}[d]$	$\mathrm{d}[e]$	$\mathrm{d}[f]$
início	—	0	∞	∞	∞	∞	∞

Início: $\mathrm{d}[a] = 0$ e todos os outros valores são ∞.



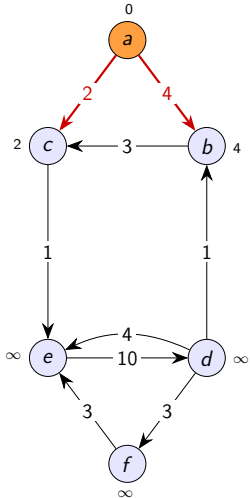
Verde: valor definitivo. **Laranja**: acabou de fechar. **Setas vermelhas**: arcos relaxados.

Dijkstra no exemplo (origem $s = a$)

iteração	escolhe u	$\mathrm{d}[a]$	$\mathrm{d}[b]$	$\mathrm{d}[c]$	$\mathrm{d}[d]$	$\mathrm{d}[e]$	$\mathrm{d}[f]$
início	—	0	∞	∞	∞	∞	∞
1	a (0)	**0**	4	2	∞	∞	∞

Fecha a (menor d); relaxa (a, b) e (a, c): $\mathrm{d}[b] = 4$, $\mathrm{d}[c] = 2$.

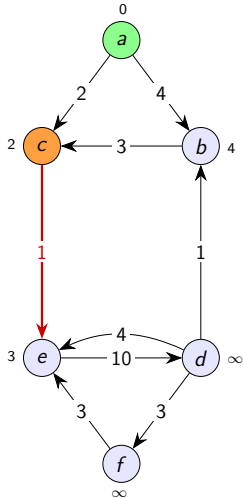
Verde: valor definitivo. **Laranja**: acabou de fechar. **Setas vermelhas**: arcos relaxados.



Dijkstra no exemplo (origem $s = a$)

iteração	escolhe u	$\mathrm{d}[a]$	$\mathrm{d}[b]$	$\mathrm{d}[c]$	$\mathrm{d}[d]$	$\mathrm{d}[e]$	$\mathrm{d}[f]$
início	—	0	∞	∞	∞	∞	∞
1	a (0)	**0**	4	2	∞	∞	∞
2	c (2)	0	4	**2**	∞	3	∞

Fecha c ($\mathrm{d} = 2$, o menor em $\overline{S}$); relaxa (c, e): $\mathrm{d}[e] = 3$.

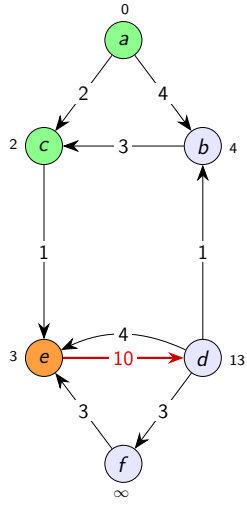


Verde: valor definitivo. **Laranja**: acabou de fechar. **Setas vermelhas**: arcos relaxados.

Dijkstra no exemplo (origem $s = a$)

iteração	escolhe u	$\mathrm{d}[a]$	$\mathrm{d}[b]$	$\mathrm{d}[c]$	$\mathrm{d}[d]$	$\mathrm{d}[e]$	$\mathrm{d}[f]$
início	—	0	∞	∞	∞	∞	∞
1	a (0)	**0**	4	2	∞	∞	∞
2	c (2)	0	4	**2**	∞	3	∞
3	e (3)	0	4	2	13	**3**	∞

Fecha e (3); relaxa (e, d): $\mathrm{d}[d] = 13$.



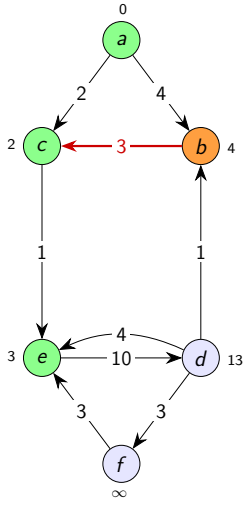
Verde: valor definitivo. **Laranja**: acabou de fechar. **Setas vermelhas**: arcos relaxados.

Dijkstra no exemplo (origem $s = a$)

iteração	escolhe u	$\mathrm{d}[a]$	$\mathrm{d}[b]$	$\mathrm{d}[c]$	$\mathrm{d}[d]$	$\mathrm{d}[e]$	$\mathrm{d}[f]$
início	—	0	∞	∞	∞	∞	∞
1	a (0)	**0**	4	2	∞	∞	∞
2	c (2)	0	4	**2**	∞	3	∞
3	e (3)	0	4	2	13	**3**	∞
4	b (4)	0	**4**	2	13	3	∞

Fecha b (4); relaxar (b, c) daria $4 + 3 = 7 > 2$: nada muda.

Verde: valor definitivo. **Laranja**: acabou de fechar. **Setas vermelhas**: arcos relaxados.

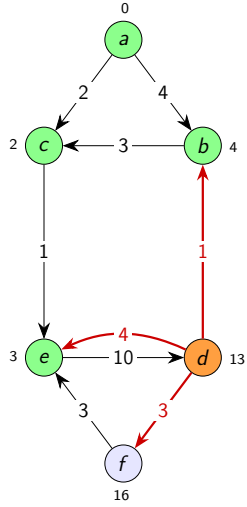


Dijkstra no exemplo (origem $s = a$)

iteração	escolhe u	$\mathrm{d}[a]$	$\mathrm{d}[b]$	$\mathrm{d}[c]$	$\mathrm{d}[d]$	$\mathrm{d}[e]$	$\mathrm{d}[f]$
início	—	0	∞	∞	∞	∞	∞
1	a (0)	**0**	4	2	∞	∞	∞
2	c (2)	0	4	**2**	∞	3	∞
3	e (3)	0	4	2	13	**3**	∞
4	b (4)	0	**4**	2	13	3	∞
5	d (13)	0	4	2	**13**	3	16

Fecha d (13); relaxa (d, f): $\mathrm{d}[f] = 16$. Os arcos (d, b) e (d, e) não melhoram nada.

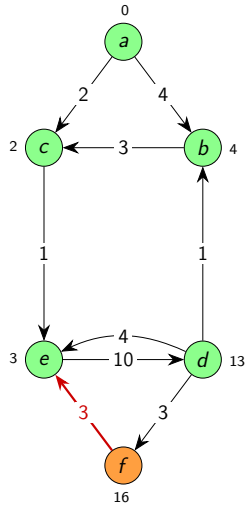
Verde: valor definitivo. **Laranja**: acabou de fechar. **Setas vermelhas**: arcos relaxados.



Dijkstra no exemplo (origem $s = a$)

iteração	escolhe u	$\mathrm{d}[a]$	$\mathrm{d}[b]$	$\mathrm{d}[c]$	$\mathrm{d}[d]$	$\mathrm{d}[e]$	$\mathrm{d}[f]$
início	—	0	∞	∞	∞	∞	∞
1	a (0)	**0**	4	2	∞	∞	∞
2	c (2)	0	4	**2**	∞	3	∞
3	e (3)	0	4	2	13	**3**	∞
4	b (4)	0	**4**	2	13	3	∞
5	d (13)	0	4	2	**13**	3	16
6	f (16)	0	4	2	13	3	**16**

Fecha f (16); (f, e) não melhora. Todos fechados. O caminho até f (via p) é a, c, e, d, f.



Verde: valor definitivo. **Laranja**: acabou de fechar. **Setas vermelhas**: arcos relaxados.

Dijkstra no exemplo (origem $s = a$)

iteração	escolhe u	$\mathrm{d}[a]$	$\mathrm{d}[b]$	$\mathrm{d}[c]$	$\mathrm{d}[d]$	$\mathrm{d}[e]$	$\mathrm{d}[f]$
início	—	0	∞	∞	∞	∞	∞
1	a (0)	**0**	4	2	∞	∞	∞
2	c (2)	0	4	**2**	∞	3	∞
3	e (3)	0	4	2	13	**3**	∞
4	b (4)	0	**4**	2	13	3	∞
5	d (13)	0	4	2	**13**	3	16
6	f (16)	0	4	2	13	3	**16**

Em negrito, o valor que acabou de ser fechado. Na iteração 4, relaxar (b, c) dá $4 + 3 = 7 > 2$: nada muda. O caminho até f (via p) é a, c, e, d, f.

Dijkstra: o algoritmo

Dijkstra(G, s)

Dado : grafo dirigido G, pesos $\rho \geq 0$, vértice s
Devolve : $\mathrm{d}[v] = \mathrm{dist}_G(s, v)$ e predecessores $\mathrm{p}[v]$

```
1 para cada v ∈ V faça  { d[v] ← ∞;  p[v] ← nil }
2 d[s] ← 0;  S ← ∅
3 enquanto S̄ ≠ ∅ faça
4 |   seja u ∈ S̄ com d[u] mínimo
5 |   insira u em S
6 |   para cada t ∈ N⁺_G(u) faça
7 |   |   Relaxação(u, t)
8 retorne d, p
```

Dijkstra: por que funciona (1/2) — o invariante

Teorema (invariante)

Antes de cada teste do laço ***enquanto****:* $\mathrm{d}[v] = \mathrm{dist}(s, v)$ *para todo* $v \in S$.

Dois fatos auxiliares, ambos consequência da desigualdade triangular:

1. **Nunca subestimamos:** $\mathrm{d}[v] \geq \mathrm{dist}(s, v)$ sempre, pois $\mathrm{d}[v]$ é o custo de um caminho real. Logo, se $\mathrm{d}[v]$ chega ao valor exato, ele congela.

Dijkstra: por que funciona (1/2) — o invariante

Teorema (invariante)

Antes de cada teste do laço ***enquanto****:* $\mathrm{d}[v] = \mathrm{dist}(s, v)$ *para todo* $v \in S$.

Dois fatos auxiliares, ambos consequência da desigualdade triangular:

1. **Nunca subestimamos:** $\mathrm{d}[v] \geq \mathrm{dist}(s, v)$ sempre, pois $\mathrm{d}[v]$ é o custo de um caminho real. Logo, se $\mathrm{d}[v]$ chega ao valor exato, ele congela.
2. **Relaxar o arco certo fecha o valor:** se $s, x_1, \ldots, x_k, v$ é caminho mínimo e $\mathrm{d}[x_k] = \mathrm{dist}(s, x_k)$, então após Relaxação$(x_k, v)$ temos $\mathrm{d}[v] = \mathrm{dist}(s, v)$ (subestrutura ótima).

Dijkstra: por que funciona (1/2) — o invariante

Teorema (invariante)

Antes de cada teste do laço ***enquanto****:* $\mathrm{d}[v] = \mathrm{dist}(s, v)$ *para todo* $v \in S$.

Dois fatos auxiliares, ambos consequência da desigualdade triangular:

1. **Nunca subestimamos:** $\mathrm{d}[v] \geq \mathrm{dist}(s, v)$ sempre, pois $\mathrm{d}[v]$ é o custo de um caminho real. Logo, se $\mathrm{d}[v]$ chega ao valor exato, ele congela.

2. **Relaxar o arco certo fecha o valor:** se $s, x_1, \ldots, x_k, v$ é caminho mínimo e $\mathrm{d}[x_k] = \mathrm{dist}(s, x_k)$, então após Relaxação$(x_k, v)$ temos $\mathrm{d}[v] = \mathrm{dist}(s, v)$ (subestrutura ótima).

A correção final sai do invariante: o laço roda $|V|$ vezes, no fim $S = V$ e portanto $\mathrm{d} = \mathrm{dist}$ em todos os vértices. Os inalcançáveis ficam com ∞.

Dijkstra: por que funciona (2/2) — esboço da prova

Suponha que o invariante falhe e seja u o primeiro vértice inserido em S com $\mathrm{d}[u] > \mathrm{dist}(s, u)$.

Dijkstra: por que funciona (2/2) — esboço da prova

Suponha que o invariante falhe e seja u o primeiro vértice inserido em S com $\mathrm{d}[u] > \mathrm{dist}(s, u)$.

1. Tome um caminho mínimo $s \rightsquigarrow u$. Ele começa em S e termina fora de S; seja (x, y) o primeiro arco que sai de S.

Dijkstra: por que funciona (2/2) — esboço da prova

Suponha que o invariante falhe e seja u o primeiro vértice inserido em S com $\mathrm{d}[u] > \mathrm{dist}(s, u)$.

1. Tome um caminho mínimo $s \rightsquigarrow u$. Ele começa em S e termina fora de S; seja (x, y) o primeiro arco que sai de S.
2. Quando x entrou em S, estava correto e o arco (x, y) foi relaxado. Pelo fato 2, $\mathrm{d}[y] = \mathrm{dist}(s, y)$.

Dijkstra: por que funciona (2/2) — esboço da prova

Suponha que o invariante falhe e seja u o primeiro vértice inserido em S com $\mathrm{d}[u] > \mathrm{dist}(s, u)$.

1. Tome um caminho mínimo $s \rightsquigarrow u$. Ele começa em S e termina fora de S; seja (x, y) o primeiro arco que sai de S.
2. Quando x entrou em S, estava correto e o arco (x, y) foi relaxado. Pelo fato 2, $\mathrm{d}[y] = \mathrm{dist}(s, y)$.
3. Como os pesos são ≥ 0, o trecho $s \rightsquigarrow y$ não é mais longo que $s \rightsquigarrow u$:

$$\mathrm{d}[y] = \mathrm{dist}(s, y) \;\leq\; \mathrm{dist}(s, u) \;<\; \mathrm{d}[u].$$

Dijkstra: por que funciona (2/2) — esboço da prova

Suponha que o invariante falhe e seja u o primeiro vértice inserido em S com $\mathrm{d}[u] > \mathrm{dist}(s, u)$.

1. Tome um caminho mínimo $s \rightsquigarrow u$. Ele começa em S e termina fora de S; seja (x, y) o primeiro arco que sai de S.
2. Quando x entrou em S, estava correto e o arco (x, y) foi relaxado. Pelo fato 2, $\mathrm{d}[y] = \mathrm{dist}(s, y)$.
3. Como os pesos são ≥ 0, o trecho $s \rightsquigarrow y$ não é mais longo que $s \rightsquigarrow u$:
$$\mathrm{d}[y] = \mathrm{dist}(s, y) \;\leq\; \mathrm{dist}(s, u) \;<\; \mathrm{d}[u].$$
4. Mas u foi escolhido com $\mathrm{d}[u]$ mínimo em $\overline{S}$, e $y \in \overline{S}$, logo $\mathrm{d}[u] \leq \mathrm{d}[y]$.

Contradição. ■

Dijkstra: complexidade

Inicialização: $O(|V|)$. Cada arco é relaxado uma vez: $O(|E|)$. O custo depende de como achar o mínimo.

Busca linear do mínimo

Em cada rodada o custo é $|\overline{S}| - 1$, e $\sum_{i=1}^{|V|}(i-1) = \frac{|V|(|V|-1)}{2}$. Total:

$$O(|V|^2 + |E|).$$

Heap binária (fila de prioridades)

$|V|$ operações `extrair-min` e no máximo $|E|$ operações `diminuir-chave`, cada uma em $O(\log |V|)$. Total:

$$O\big((|V| + |E|) \log |V|\big).$$

Bellman–Ford: motivação

- Aceita arcos com peso negativo.
- Pode haver um **circuito negativo**: um circuito orientado cuja soma dos pesos é < 0.
- Num circuito negativo, a cada volta o comprimento diminui, então a distância não está bem definida ($-\infty$).

O que o algoritmo devolve

Falso se existe um circuito negativo alcançável a partir de s. Caso contrário, *verdadeiro* e $\mathrm{d}[v] = \mathrm{dist}(s, v)$ para todo v.

Bellman–Ford: a ideia

Se não há circuito negativo, um caminho mínimo $s = x_0, x_1, \ldots, x_k = v$ não repete vértices, logo $k \leq |V| - 1$.

Repetir $|V| - 1$ vezes: relaxar *todos* os arcos, em qualquer ordem

- Rodada 1: (x_0, x_1) é relaxado, então $\mathrm{d}[x_1]$ fica correto.
- Rodada 2: (x_1, x_2) é relaxado, então $\mathrm{d}[x_2]$ fica correto.
- ...
- Rodada i: $\mathrm{d}[x_i]$ fica correto.

A informação correta avança pelo menos um arco por rodada, como uma onda saindo de s. Após $|V| - 1$ rodadas, qualquer caminho mínimo está resolvido.

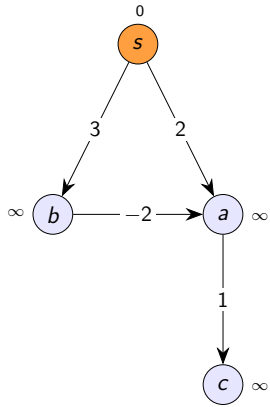
Detecção: se ainda existir um arco (u, w) com $\mathrm{d}[w] > \mathrm{d}[u] + \rho(u, w)$, algo ainda melhora, o que só ocorre com um circuito negativo.

Bellman–Ford no exemplo com pesos negativos

Arcos na ordem pior possível: $(a, c), (b, a), (s, a), (s, b)$.
Como $|V| = 4$, são $|V| - 1 = 3$ rodadas.

	$\mathrm{d}[s]$	$\mathrm{d}[a]$	$\mathrm{d}[b]$	$\mathrm{d}[c]$	arcos que melhoram
início	**0**	∞	∞	∞	—

Início: $\mathrm{d}[s] = 0$, os outros valores são ∞.

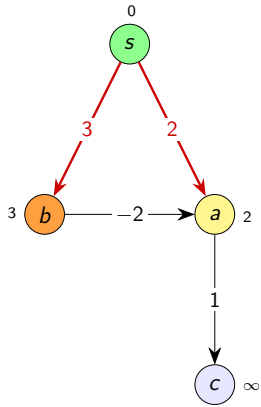


Azul: ainda ∞. **Amarelo**: valor provisório. **Laranja**: ficou definitivo agora. **Verde**: definitivo. **Setas vermelhas**: arcos que melhoram.

Bellman–Ford no exemplo com pesos negativos

Arcos na ordem pior possível: $(a, c), (b, a), (s, a), (s, b)$.
Como $|V| = 4$, são $|V| - 1 = 3$ rodadas.

	$\mathrm{d}[s]$	$\mathrm{d}[a]$	$\mathrm{d}[b]$	$\mathrm{d}[c]$	arcos que melhoram
início	**0**	∞	∞	∞	—
rodada 1	0	2	**3**	∞	$(s, a), (s, b)$



Rodada 1: (a, c) e (b, a) não fazem nada, pois $\mathrm{d}[a] = \mathrm{d}[b] = \infty$; (s, a) dá $\mathrm{d}[a] = 2$ e (s, b) dá $\mathrm{d}[b] = 3$. Agora b está correto.

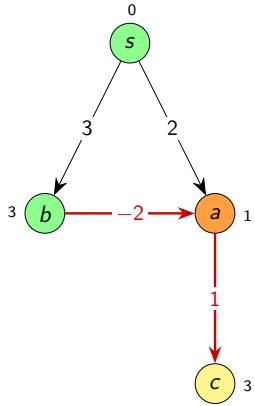
Azul: ainda ∞. **Amarelo**: valor provisório. **Laranja**: ficou definitivo agora. **Verde**: definitivo. **Setas vermelhas**: arcos que melhoram.

Bellman–Ford no exemplo com pesos negativos

Arcos na ordem pior possível: $(a, c), (b, a), (s, a), (s, b)$.
Como $|V| = 4$, são $|V| - 1 = 3$ rodadas.

	$\mathrm{d}[s]$	$\mathrm{d}[a]$	$\mathrm{d}[b]$	$\mathrm{d}[c]$	arcos que melhoram
início	**0**	∞	∞	∞	—
rodada 1	0	2	**3**	∞	$(s, a), (s, b)$
rodada 2	0	**1**	3	3	$(a, c), (b, a)$

Rodada 2: (a, c) dá $\mathrm{d}[c] = 2 + 1 = 3$; depois (b, a) dá $\mathrm{d}[a] = 3 - 2 = 1$. Agora a está correto.



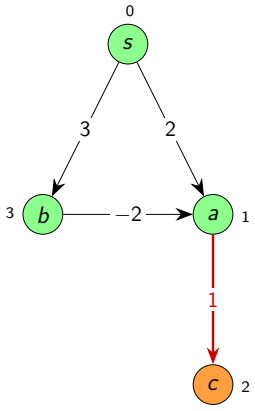
Azul: ainda ∞. **Amarelo**: valor provisório. **Laranja**: ficou definitivo agora. **Verde**: definitivo. **Setas vermelhas**: arcos que melhoram.

Bellman–Ford no exemplo com pesos negativos

Arcos na ordem pior possível: $(a, c), (b, a), (s, a), (s, b)$.
Como $|V| = 4$, são $|V| - 1 = 3$ rodadas.

	$\mathrm{d}[s]$	$\mathrm{d}[a]$	$\mathrm{d}[b]$	$\mathrm{d}[c]$	arcos que melhoram
início	**0**	∞	∞	∞	—
rodada 1	0	2	**3**	∞	$(s, a), (s, b)$
rodada 2	0	**1**	3	3	$(a, c), (b, a)$
rodada 3	0	1	3	**2**	(a, c)

Rodada 3: (a, c) dá $\mathrm{d}[c] = 1 + 1 = 2$. Agora c está correto: o caminho mínimo s, b, a, c foi "descoberto" um arco por rodada.



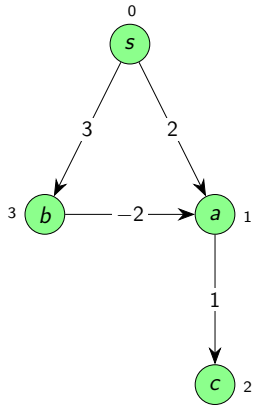
Azul: ainda ∞. **Amarelo**: valor provisório. **Laranja**: ficou definitivo agora. **Verde**: definitivo. **Setas vermelhas**: arcos que melhoram.

Bellman–Ford no exemplo com pesos negativos

Arcos na ordem pior possível: $(a, c), (b, a), (s, a), (s, b)$.
Como $|V| = 4$, são $|V| - 1 = 3$ rodadas.

	$\mathrm{d}[s]$	$\mathrm{d}[a]$	$\mathrm{d}[b]$	$\mathrm{d}[c]$	arcos que melhoram
início	**0**	∞	∞	∞	—
rodada 1	0	2	**3**	∞	$(s, a), (s, b)$
rodada 2	0	**1**	3	3	$(a, c), (b, a)$
rodada 3	0	1	3	**2**	(a, c)
verificação	0	1	3	2	nenhum

Verificação final: nenhum arco satisfaz $\mathrm{d}[w] > \mathrm{d}[u] + \rho(u, w)$, então o algoritmo devolve *verdadeiro* e $\mathrm{d} = \mathrm{dist}$.



Azul: ainda ∞. **Amarelo**: valor provisório. **Laranja**: ficou definitivo agora. **Verde**: definitivo. **Setas vermelhas**: arcos que melhoram.

Exercício

Simule Dijkstra no exemplo anterior e justifique o motivo do erro.

Bellman–Ford: o algoritmo

Bellman–Ford(G, s)

Dado : $G = (V, E, \rho)$ dirigido, ρ arbitrário, e $s \in V$
Devolve : *falso* se há circuito negativo alcançável de s; senão *verdadeiro*, $\mathrm{d} = \mathsf{dist}_G(s, \cdot)$ e p

```
1 para cada v ∈ V faça  { d[v] ← ∞;  p[v] ← nil }
2 d[s] ← 0
3 para cont ← 1 to |V| − 1 faça
4 |   para cada (u, w) ∈ E faça
5 |   |   Relaxação(u, w)
6 para cada (u, w) ∈ E faça
7 |   se d[w] > d[u] + ρ(u, w) então  retorne falso
8 retorne verdadeiro, d, p
```

Bellman–Ford: por que funciona (1/3) — sem circuito negativo

Dois fatos, valem sempre:

(a) **Nunca subestimamos.** $\mathrm{d}[v]$ é ∞ ou o peso de algum passeio de s a v. Sem circuitos negativos, todo passeio pesa pelo menos $\mathrm{dist}(s, v)$ (um passeio é um caminho mais circuitos de peso ≥ 0). Logo $\mathrm{d}[v] \geq \mathrm{dist}(s, v)$.

(b) **A onda avança.** Se $P = x_0, \ldots, x_k$ é um caminho com $x_0 = s$, então após a rodada i vale $\mathrm{d}[x_i] \leq c(x_0, \ldots, x_i)$.

Bellman–Ford: por que funciona (2/3) — sem circuito negativo

Prova de (b), por indução em i.

Para $i = 0$, $\mathrm{d}[s] \leq 0$.

Na rodada i, o arco (x_{i-1}, x_i) é relaxado e, como $\mathrm{d}[x_{i-1}] \leq c(x_0, \ldots, x_{i-1})$ pela hipótese de indução,

$$\mathrm{d}[x_i] \leq \mathrm{d}[x_{i-1}] + \rho(x_{i-1}, x_i) \leq c(x_0, \ldots, x_i).$$

(Os valores de d nunca aumentam, então essa desigualdade se mantém.)

Aplicando (b) a um caminho mínimo, com $k \leq |V| - 1$, obtemos $\mathrm{d}[v] \leq \mathrm{dist}(s, v)$. Com (a), $\mathrm{d}[v] = \mathrm{dist}(s, v)$, e a verificação final não encontra nenhum arco a melhorar.

Bellman–Ford: por que funciona (2/2) — com circuito negativo

Seja $C = v_0, v_1, \ldots, v_{k-1}, v_0$ um circuito negativo alcançável (e $v_k = v_0$). Todos os v_i têm $\mathrm{d}[v_i] < \infty$.

Suponha, por absurdo, que o algoritmo devolve verdadeiro. Então nenhum arco falha no teste final:

$$\mathrm{d}[v_i] \leq \mathrm{d}[v_{i-1}] + \rho(v_{i-1}, v_i), \qquad i = 1, \ldots, k.$$

Somando as k desigualdades:

$$\sum_{i=1}^{k} \mathrm{d}[v_i] \;\leq\; \sum_{i=1}^{k} \mathrm{d}[v_{i-1}] \;+\; \sum_{i=1}^{k} \rho(v_{i-1}, v_i).$$

Como $v_k = v_0$, as duas somas de d são iguais e finitas, e se cancelam:

$$0 \;\leq\; \sum_{i=1}^{k} \rho(v_{i-1}, v_i) = c(C) < 0.$$

Absurdo. Logo o algoritmo devolve falso. ■

Bellman–Ford: complexidade

- Inicialização: $\Theta(|V|)$.
- $|V| - 1$ rodadas, cada uma com $|E|$ relaxações em $O(1)$: $O(|V||E|)$.
- Verificação final: $O(|E|)$.

Tempo total

$$O(|V||E|).$$

- Se numa rodada nenhum d mudar, as seguintes também não mudam: dá para parar mais cedo.
- É mais lento que Dijkstra, e esse é o preço de aceitar pesos negativos e detectar circuitos negativos.

Dijkstra × Bellman–Ford

	Dijkstra	**Bellman–Ford**
Pesos	$\rho \geq 0$	quaisquer
Circuito negativo	não se aplica	detecta
Estratégia	guloso: fecha um vértice por vez	relaxa tudo $\lvert V\rvert - 1$ vezes
Cada arco é relaxado	uma vez	$\lvert V\rvert - 1$ vezes
Tempo (simples)	$O(\lvert V\rvert^2 + \lvert E\rvert)$	$O(\lvert V\rvert \lvert E\rvert)$
Tempo (heap)	$O\big((\lvert V\rvert + \lvert E\rvert) \log \lvert V\rvert\big)$	—